

Gradient Descent

Machine Learning & IoT Lab^{*a}

^aHo Chi Minh City University of Technology (HCMUT)

TA: Nguyễn Minh Khánh

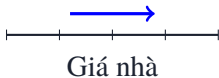
khanhnguyen2712.work@gmail.com

1. Câu hỏi: mô hình học từ lỗi thế nào?
2. Công cụ toán: đạo hàm, gradient, chain rule.
3. Thuật toán Gradient Descent: Taylor bậc nhất và công thức update.
4. Hai mô hình Machine Learning: Linear và Logistic Regression.
5. Loss functions: MSE, MAE, Cross-Entropy.
6. Suy diễn gradient: công thức cuối cho w, b .
7. Bài tập tính tay: 2 epoch cho 1 mẫu dữ liệu.
8. Mở rộng: BatchGD/SGD/Mini-batchGD, optimizer

Regression

2.8

Từ diện tích căn nhà, dự đoán một **giá trị liên tục**.



Classification

0.92

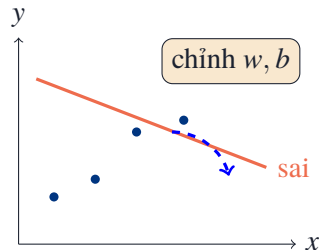
Từ nội dung email, dự đoán **xác suất spam**.



Điểm chung Cả hai mô hình đều bắt đầu với tham số chưa phù hợp và có thể dự đoán sai.

- ▶ Loss cho biết mô hình đang sai **bao nhiêu**.
- ▶ Nhưng ta còn cần biết phải chỉnh w, b theo **hướng nào**.
- ▶ Và mỗi lần nên chỉnh **bao nhiêu**.

Câu hỏi trung tâm Máy tính biến một dự đoán sai thành dự đoán gần đúng?





1. Tạo dự đoán từ tham số hiện tại.
2. Đo sai số bằng một loss.
3. Tính gradient theo w, b .
4. Cập nhật và lặp lại.

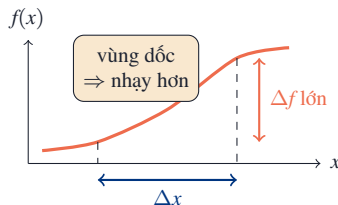
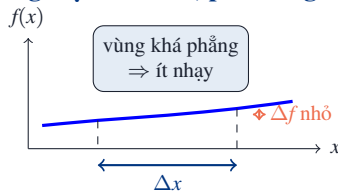
Cuối buổi, bạn có thể

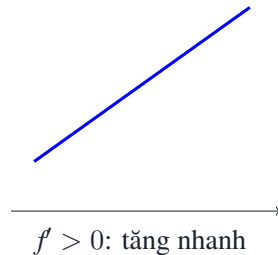
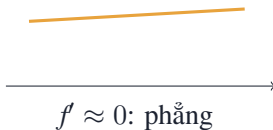
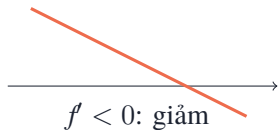
- ▶ hiểu được bản chất và cách hoạt động của Gradient Descent;
- ▶ tính tay một iteration;
- ▶ hiện thực vòng lặp bằng NumPy.

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- ▶ Không chỉ là một quy tắc tính toán.
- ▶ Nó trả lời: tăng input một lượng rất nhỏ thì output đổi bao nhiêu?

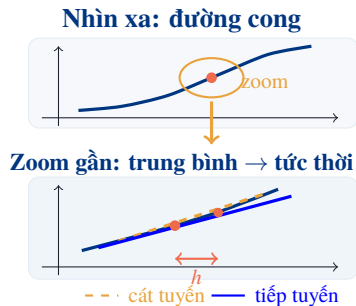
Cùng một bước Δx , phản ứng có thể khác





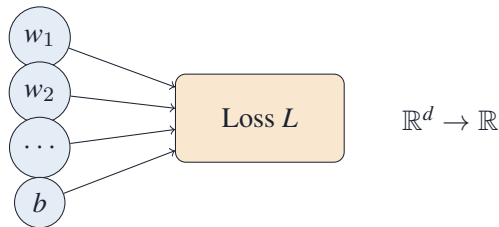
Hai thông tin Dấu cho biết chiều thay đổi; $|f'(x)|$ cho biết mức thay đổi mạnh hay yếu.

- ▶ Cát tuyến là đường thẳng nối **hai điểm** trên đồ thị.
- ▶ Độ dốc của cát tuyến cho biết mức thay đổi **trung bình**.
- ▶ Khi hai điểm tiến sát nhau, cát tuyến dần thành tiếp tuyến.
- ▶ Độ dốc của tiếp tuyến chính là **đạo hàm tại điểm đó**.



$$L(w, b) \longrightarrow L(\theta), \theta \in \mathbb{R}^d$$

- ▶ Linear Regression đơn giản đã có w và b .
- ▶ Neural network có thể có hàng triệu tham số.
- ▶ Mỗi tham số là một chiều trong không gian loss.

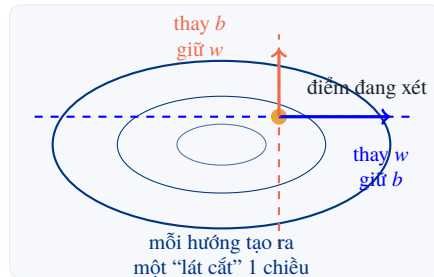


$$f(w, b) = w^2 + 2b^2$$

$$\frac{\partial f}{\partial w} = 2w, \quad \frac{\partial f}{\partial b} = 4b$$

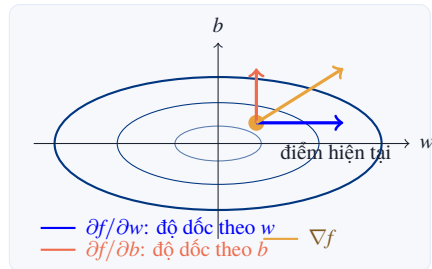
- ▶ $\partial f / \partial w$: cho w thay đổi, giữ b cố định.
- ▶ $\partial f / \partial b$: cho b thay đổi, giữ w cố định.

Mỗi đạo hàm riêng là **độ dốc** trên một lát cắt của bề mặt.



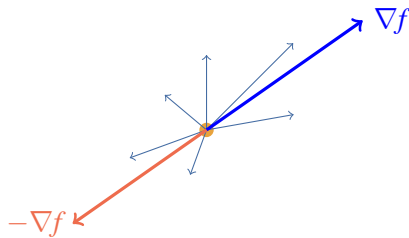
$$\nabla f(w, b) = \begin{bmatrix} \partial f / \partial w \\ \partial f / \partial b \end{bmatrix}$$

- ▶ Mỗi dòng là một câu trả lời: nếu nhích tham số này, f đổi bao nhiêu?
- ▶ Gradient gom các câu trả lời đó thành một vector.
- ▶ Vector này cho ta biết nên nhìn về phía nào trong không gian tham số.



$$D_{\mathbf{u}}f(\mathbf{x}) = \nabla f(\mathbf{x})^T \mathbf{u}$$

- ▶ Cùng hướng gradient: tăng nhanh nhất.
- ▶ Ngược hướng gradient: giảm nhanh nhất theo xấp xỉ bậc nhất.



“la bàn” độ dốc



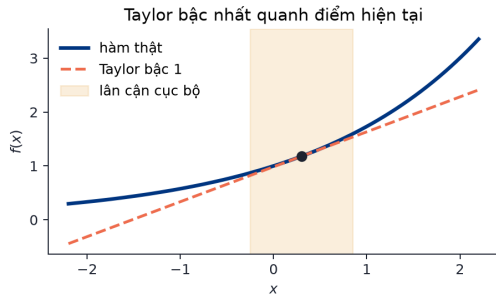
đạo hàm truyền ngược

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w}$$

Cầu nối Backpropagation sau này chính là chain rule được tổ chức hiệu quả trên một graph lớn.

$$L(\theta + \Delta\theta) \approx L(\theta) + \nabla L(\theta)^\top \Delta\theta$$

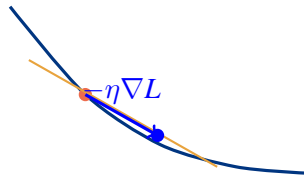
- ▶ Thay bề mặt phức tạp bằng mặt phẳng cục bộ.
- ▶ Xấp xỉ tốt khi bước $\Delta\theta$ đủ nhỏ.



Chọn $\Delta\theta = -\eta\nabla L(\theta)$, với $\eta > 0$:

$$L(\theta + \Delta\theta) \approx L(\theta) - \eta\|\nabla L(\theta)\|^2.$$

Kết luận Nếu bước đủ nhỏ và gradient khác 0, mô hình cục bộ dự đoán loss sẽ giảm.



$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

Ý nghĩa từng thành phần

- ▶ θ_t : tham số hiện tại.
- ▶ θ_{t+1} : tham số sau khi cập nhật.
- ▶ $L(\theta_t)$: loss tại vị trí hiện tại.
- ▶ $\nabla L(\theta_t)$: hướng loss tăng nhanh nhất.

Ý nghĩa của bước cập nhật

- ▶ Dấu trừ: đi ngược hướng tăng để giảm loss.
- ▶ η : learning rate, quyết định bước đi dài hay ngắn.
- ▶ $\eta \nabla L(\theta_t)$: lượng điều chỉnh tham số.

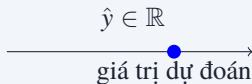


1. $\hat{y} = \text{predict}(x; w_t, b_t)$
2. $L = \text{loss}(\hat{y}, y)$
3. $g_w = \partial L / \partial w, g_b = \partial L / \partial b$
4. $w_{t+1} = w_t - \eta g_w, b_{t+1} = b_t - \eta g_b$

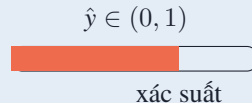
Quy ước quan trọng

Mọi giá trị forward và gradient trong iteration t đều dùng cùng w_t, b_t . Chỉ cập nhật ở bước cuối.

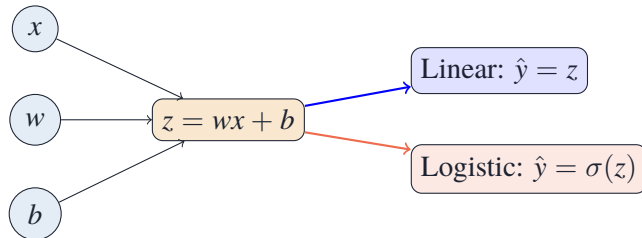
Linear Regression



Logistic Regression



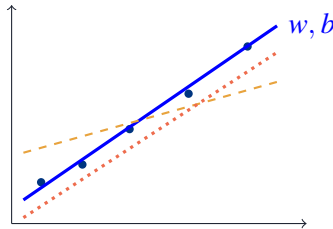
Điểm chung Cả hai đều bắt đầu bằng linear score $z = wx + b$.



- ▶ w : mức ảnh hưởng của đặc trưng x .
- ▶ b : baseline khi $x = 0$.

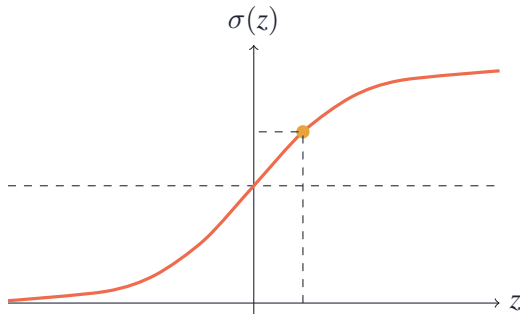
$$\hat{y} = wx + b$$

- ▶ w đổi độ dốc.
- ▶ b tịnh tiến đường dự đoán.



$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- ▶ Mọi score được nén vào khoảng $(0, 1)$.
- ▶ Threshold chỉ cần khi chuyển xác suất thành nhãn.



Một mẫu

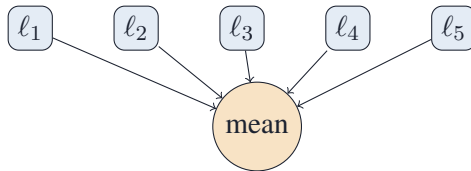
$$\ell_i = \ell(\hat{y}_i, y_i)$$

Mỗi dự đoán có một loss riêng.

Một tập gồm n mẫu

$$L = \frac{1}{n} \sum_{i=1}^n \ell_i$$

Training thường tối thiểu hóa loss trung bình.

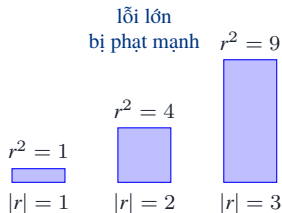


$$r_i = \hat{y}_i - y_i, \quad \text{MSE} = \frac{1}{n} \sum_{i=1}^n r_i^2$$

- ▶ Dùng nhiều trong **Regression**.
- ▶ Trên một mẫu: lấy lỗi $r = \hat{y} - y$, rồi bình phương.
- ▶ Sai càng lớn thì bị phạt càng mạnh.

Ví dụ một mẫu

$$y = 10, \quad \hat{y} = 8$$
$$r = 8 - 10 = -2, \quad r^2 = 4$$



$$r_i = \hat{y}_i - y_i, \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |r_i|$$

- ▶ Cũng dùng cho **Regression**.
- ▶ Trên một mẫu: chỉ lấy khoảng cách giữa dự đoán và giá trị thật.
- ▶ Lỗi tăng gấp đôi thì loss cũng tăng gấp đôi.

So sánh nhanh với MSE

$ r $	1	2	3
MAE	1	2	3
MSE	1	4	9

- ▶ MAE dễ đọc: “trung bình lệch bao nhiêu đơn vị?”
- ▶ MSE nhấn mạnh các lỗi lớn hơn MAE.
- ▶ Trong bài tính gradient sau, ta dùng loss bình phương vì đạo hàm gọn hơn.

$$\ell = -y \log p - (1 - y) \log(1 - p)$$

- ▶ Dùng cho **Classification** nhị phân.
- ▶ p : xác suất mô hình dự đoán cho lớp 1.
- ▶ Nếu $y = 1$: $\ell = -\log p$.
- ▶ Nếu $y = 0$: $\ell = -\log(1 - p)$.

Cách đọc

y	p	ℓ
1	0.9	$-\log(0.9)$ nhỏ
1	0.1	$-\log(0.1)$ lớn
0	0.9	$-\log(0.1)$ lớn

Cách hiểu Đưa xác suất cao cho đáp án đúng thì loss nhỏ; tự tin vào đáp án sai thì loss lớn.

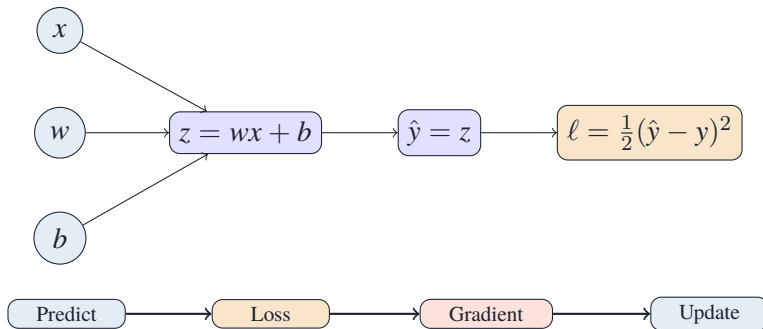
Linear — một mẫu

$$\ell_{\text{lin}} = \frac{1}{2}(\hat{y} - y)^2$$

Logistic — một mẫu

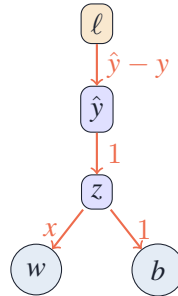
$$\ell_{\text{log}} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

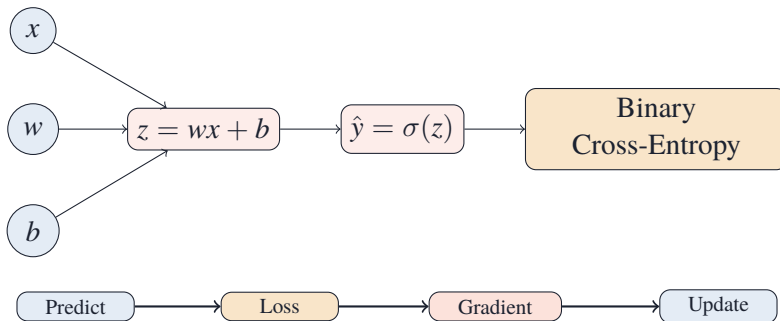
Lưu ý Prediction và loss phải phù hợp với loại đầu ra của mô hình.



Forward Đi từ trái sang phải để tạo prediction và loss bằng tham số hiện tại.

$$\frac{\partial \ell}{\partial w} = \underbrace{(\hat{y} - y)}_{\partial \ell / \partial \hat{y}} \underbrace{(1)}_{\partial \hat{y} / \partial z} \underbrace{x}_{\partial z / \partial w} = (\hat{y} - y)x$$
$$\frac{\partial \ell}{\partial b} = (\hat{y} - y).$$

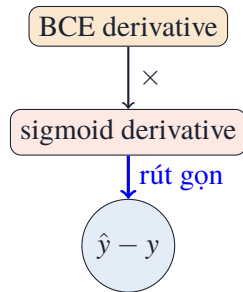




Khác Linear Logistic thêm sigmoid trước khi loss đánh giá xác suất.

$$\frac{\partial \ell}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}, \quad \frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y}).$$

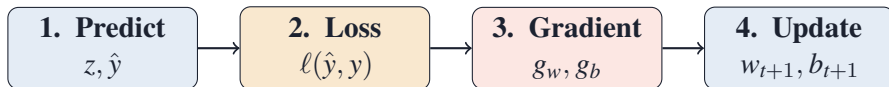
$$\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} = \boxed{\hat{y} - y}.$$



	Linear Regression	Logistic Regression
Prediction	$\hat{y} = wx + b$	$\hat{y} = \sigma(wx + b)$
Loss một mẫu	$\frac{1}{2}(\hat{y} - y)^2$	BCE
$\partial \ell / \partial w$	$(\hat{y} - y)x$	$(\hat{y} - y)x$
$\partial \ell / \partial b$	$\hat{y} - y$	$\hat{y} - y$

Điểm đặc biệt Công thức cuối giống nhau, nhưng được suy ra từ hai prediction–loss pair khác nhau.





Quy tắc làm bài

Dùng w_t, b_t xuyên suốt ba bước đầu; chỉ làm tròn Logistic tới 4 chữ số; cập nhật đồng thời w, b .

$$x = 1, \quad y = 1, \quad w_0 = 0, \quad b_0 = 0, \quad \eta = 0.1, \quad \hat{y} = wx + b, \quad \ell = \frac{1}{2}(\hat{y} - y)^2$$

Epoch	$\hat{y}$	ℓ	g_w	g_b	w_{new}	b_{new}
1	_____	_____	_____	_____	_____	_____
2	_____	_____	_____	_____	_____	_____

$$x = 1, \quad y = 1, \quad w_0 = b_0 = 0, \quad \eta = 0.1$$

$$\ell = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

$$\hat{y} = \sigma(z), \quad z = wx + b$$

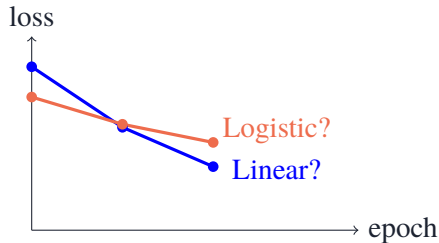
$$g_w = (\hat{y} - y)x, \quad g_b = \hat{y} - y$$

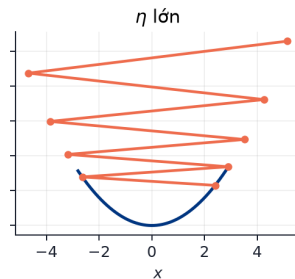
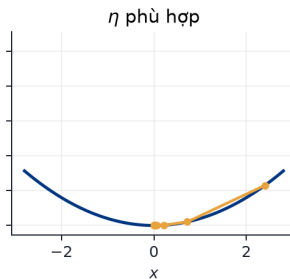
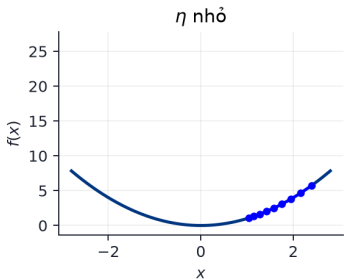
Epoch	z	$\hat{y}$	ℓ	g_w	g_b	w_{new}	b_{new}
1	_____	_____	_____	_____	_____	_____	_____
2	_____	_____	_____	_____	_____	_____	_____

Máy tính và tự kiểm tra

Được dùng máy tính cho e^{-z} , $\log$; hiển thị 4 chữ số. Với $y = 1$, kỳ vọng $\hat{y}$ tăng và BCE giảm sau mỗi epoch.

1. Ở epoch 1, hai mô hình bắt đầu với dự đoán nào?
2. Sau mỗi epoch, loss có giảm đúng như kỳ vọng không?
3. Sang epoch 2, độ lớn của g_w, g_b thay đổi ra sao?
4. Linear và Logistic cùng cập nhật w, b , nhưng vì sao số tính ra khác nhau?





η quá nhỏ

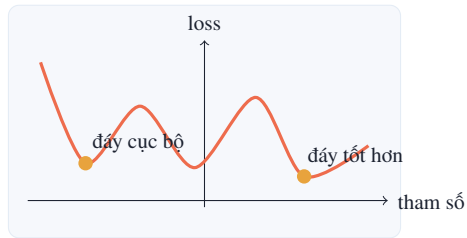
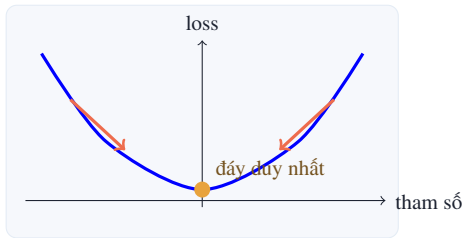
Đúng hướng nhưng đi rất chậm.

η phù hợp

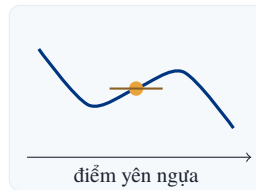
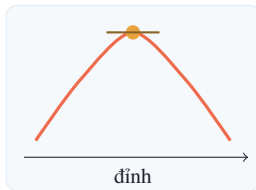
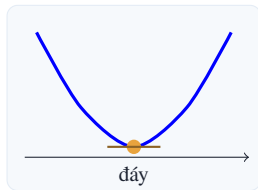
Loss giảm ổn định qua từng bước.

η quá lớn

Dễ vượt quá vùng tốt, dao động hoặc phân kỳ.

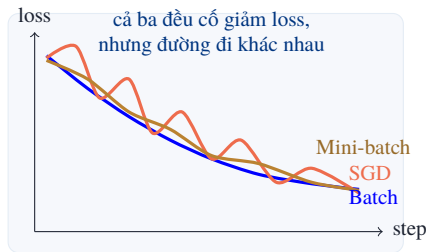


Ý chính Hàm lồi dễ tối ưu hơn: nếu đã xuống đáy cục bộ thì đó cũng là đáy tốt nhất toàn cục.



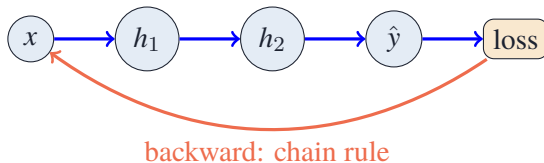
- ▶ Gradient bằng 0 chỉ nói: quanh điểm đó, mặt loss đang **phẳng theo bậc nhất**.
- ▶ Muốn biết đó là đáy, đỉnh hay yên ngựa, phải nhìn hình dạng xung quanh.

Cách	Mỗi update dùng	Cảm giác khi train
Batch GD	Toàn bộ dataset	Loss đi mượt, nhưng mỗi bước tốn thời gian và bộ nhớ.
SGD	Một mẫu	Bước rất nhanh và ít bộ nhớ, nhưng loss dao động mạnh.
Mini-batch GD	Một nhóm mẫu	Cân bằng: nhanh hơn Batch, ổn định hơn SGD.



Ý chính

Khác nhau không nằm ở công thức update, mà ở **cách ước lượng gradient** từ dữ liệu.

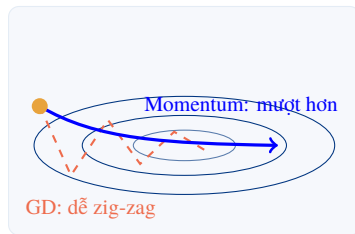


forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ optimizer step

Mối quan hệ Backpropagation lan truyền ngược sai số từ loss để tính đạo hàm bằng Chain Rule. Còn optimizer sẽ dùng Gradient Descent và các biến thể tương tự để cập nhật trọng số.

$$v_t = \beta v_{t-1} + (1 - \beta)g_t, \quad \theta_{t+1} = \theta_t - \eta v_t$$

- ▶ GD thường chỉ nhìn gradient hiện tại g_t .
- ▶ Momentum nhớ hướng đi gần đây qua v_t .
- ▶ Nếu nhiều bước cùng hướng, nó đi nhanh hơn; nếu zig-zag, các dao động triệt bớt nhau.



So với GD truyền thống

Ưu: giảm zig-zag, tăng tốc trong thung lũng hẹp.

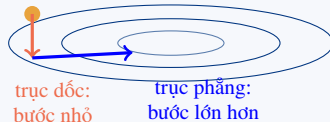
Nhược: thêm β ; quán tính quá lớn có thể vượt quá điểm tốt.

$$s_t = \rho s_{t-1} + (1 - \rho) g_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon}$$

- ▶ s_t nhớ độ lớn bình phương của gradient.
- ▶ Chiều nào gradient thường lớn: chia nhiều hơn $\Rightarrow$ bước nhỏ lại.
- ▶ Chiều nào gradient nhỏ/phẳng: bước tương đối lớn hơn.

cùng learning rate,
nhưng scale khác theo từng chiều

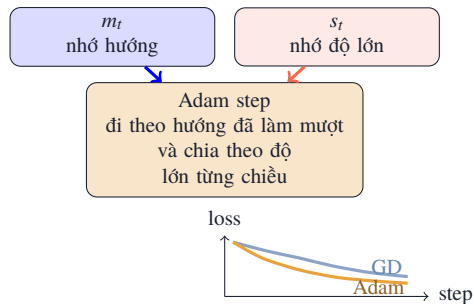


So với GD truyền thống

Ưu: bớt nhảy khi các chiều có scale gradient rất khác nhau.
Nhược: thêm ρ, ϵ ; có thể cần chỉnh η cẩn thận.

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\s_t &= \beta_2 s_{t-1} + (1 - \beta_2) g_t^2 \\ \theta_{t+1} &= \theta_t - \eta \frac{m_t}{\sqrt{s_t} + \epsilon}\end{aligned}$$

- ▶ m_t : nhớ hướng trung bình; s_t : nhớ độ lớn gradient.
- ▶ So với GD: thường nhanh/ổn định hơn, nhưng có nhiều siêu tham số hơn.



	Dùng g_t	Nhớ hướng	Scale từng chiều
GD	✓	—	—
Momentum	✓	✓	—
RMSProp	✓	—	✓
Adam	✓	✓	✓

- ▶ Momentum: đường đi mượt hơn.
- ▶ RMSProp: bước đi thích nghi theo từng tham số.
- ▶ Adam: kết hợp cả hai ý tưởng.



Thực hành Optimizer giúp bước update thông minh hơn, nhưng vẫn phải theo dõi train/validation loss.

Bức tranh lớn

- ▶ Mô hình học bằng vòng lặp:
Predict $\rightarrow$ Loss $\rightarrow$ Gradient $\rightarrow$ Update.
- ▶ Loss biến mức sai thành một con số để tối ưu.
- ▶ Gradient cho biết cần chỉnh tham số như thế nào để giảm loss.

Công thức lỗi

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

- ▶ Dấu trừ: đi ngược hướng loss tăng.
- ▶ η : độ dài bước đi.
- ▶ Sau mỗi update, loss là tín hiệu cho biết bước đi có ổn không.

Nếu training không ổn Đừng đoán mò: kiểm tra lần lượt prediction, loss, gradient, learning rate và optimizer!

Thank You

Acknowledgements

The authors gratefully acknowledge ML&IoT Lab for their support and guidance.